

Lösung_Python_Übung_1

November 8, 2024

1 Datenauswertung mit Python - Übung 1

1.1 Aufgabe 1: Verdünnungsreihe

1.1.1 Funktionen: Verdünnung

Berechne aus der gegebenen Konzentration der Stammlösung, der Konzentration der verdünnten Säure und dem Zielvolumen **die Menge and Stammlösung** die benötigt wird. Nutze dazu eine **Funktion** `get_dilution`.

```
[1]: def get_dilution(c_start, c_target, v_target):  
    """  
    Berechnet die nötige Verdünnung einer Ausgangslösung  
    c_start: Konzentration Stammlösung  
    c_target: Konzentration Ziellösung  
    v_target: Volumen der Ziellösung  
    Output:  
    v_start: Nötiges Volumen der Stammlösung  
    v_water: Nötiges Verdünnungsvolumen des Wassers  
    """  
  
    n_acid = v_target * c_target  
    v_start = n_acid / c_start  
    v_water = v_target - v_start  
  
    return v_start, v_water  
  
# Input Variablen  
concentration_available = 12.39 # Konzentration Stammlösung, mol/L  
concentration_target = 0.2 # Zielkonzentration, mol/L  
volume_target = 1.0 # Zielvolumen, L  
  
v_start, v_water = get_dilution(concentration_available,  
                                 concentration_target, volume_target)  
  
print(f"Für {volume_target} L einer Säure von {concentration_target} mol/L " \  
      f"müssen {v_start} L Säure und {v_water} L Wasser gemischt werden.")
```

Für 1.0 L einer Säure von 0.2 mol/L müssen 0.016142050040355124 L Säure und 0.9838579499596449 L Wasser gemischt werden.

1.1.2 For-Loops: Berechnung mehrerer Zielkonzentrationen

Wiederhole die Rechnung aus der vorigen Aufgabe für alle Zielkonzentrationen in der Liste `concentrations` in einem **for-Loop**. Nutze dabei die Funktion `get_dilution` innerhalb des Loops.

```
[12]: concentration_available = 12.39 # Konzentration Stammlösung, mol/L
volume_target = 1 # Zielvolumen, L
concentrations = [0.1, 0.5, 0.75, 1, 5, 10] # Zielkonzentration, mol/L

for c in concentrations:
    v_start, v_water = get_dilution(concentration_available,
                                     c, volume_target)

    print(f"Für {volume_target} L einer Säure von {c} mol/L " \
          f"müssen {v_start} L Säure und {v_water} L Wasser gemischt werden.")
```

Für 1 L einer Säure von 0.1 mol/L müssen 0.008071025020177562 L Säure und 0.9919289749798225 L Wasser gemischt werden.

Für 1 L einer Säure von 0.5 mol/L müssen 0.04035512510088781 L Säure und 0.9596448748991122 L Wasser gemischt werden.

Für 1 L einer Säure von 0.75 mol/L müssen 0.06053268765133172 L Säure und 0.9394673123486683 L Wasser gemischt werden.

Für 1 L einer Säure von 1 mol/L müssen 0.08071025020177562 L Säure und 0.9192897497982244 L Wasser gemischt werden.

Für 1 L einer Säure von 5 mol/L müssen 0.4035512510088781 L Säure und 0.5964487489911219 L Wasser gemischt werden.

Für 1 L einer Säure von 10 mol/L müssen 0.8071025020177562 L Säure und 0.19289749798224376 L Wasser gemischt werden.

1.1.3 Bedingungen: Schließe zu hohe Konzentrationen aus

Wenn die Zielkonzentration höher ist als die der Stammlösung, ist es nicht möglich diese durch Verdünnung herzustellen. Schließe diese Fälle aus indem du in den Code aus der vorigen Aufgabe eine **if/else** Bedingung einbaust, die prüft ob die Zielkonzentration höher ist als die der Stammlösung.

```
[14]: concentration_available = 12.39 # mol/L
volume_target = 1 # L
concentrations = [0.1, 0.5, 0.75, 1, 5, 10, 15] # mol/L

for c in concentrations:
    if c > concentration_available:
        print(f"Eine Säure mit {c} mol/L kann nicht aus einer Lösung mit
        ↳{concentration_available} erstellt werden.")
    else:
        v_start, v_water = get_dilution(concentration_available,
```

```

c, volume_target)

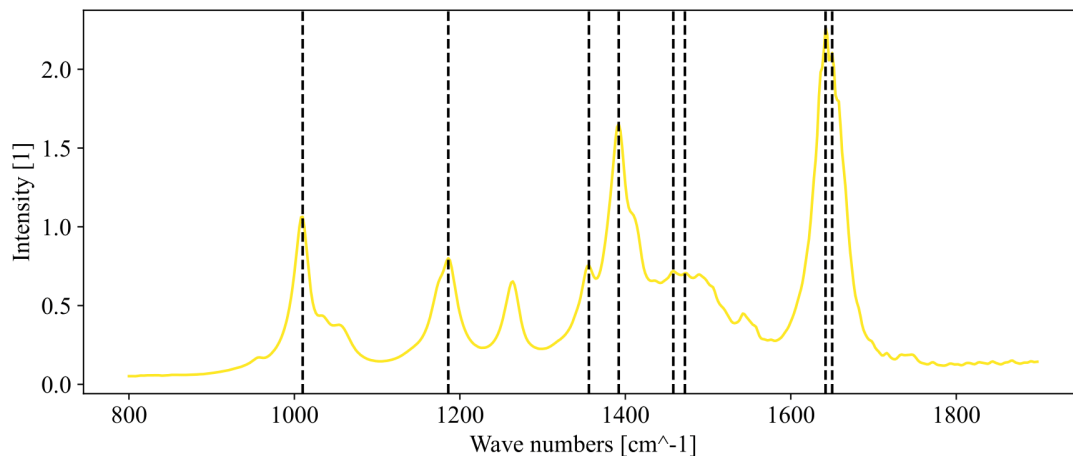
print(f"Für {volume_target} L einer Säure von {c} mol/L " \
      f"müssen {v_start} L Säure und {v_water} L Wasser gemischt werden.
↪")

```

Für 1 L einer Säure von 0.1 mol/L müssen 0.008071025020177562 L Säure und 0.9919289749798225 L Wasser gemischt werden.
 Für 1 L einer Säure von 0.5 mol/L müssen 0.04035512510088781 L Säure und 0.9596448748991122 L Wasser gemischt werden.
 Für 1 L einer Säure von 0.75 mol/L müssen 0.06053268765133172 L Säure und 0.9394673123486683 L Wasser gemischt werden.
 Für 1 L einer Säure von 1 mol/L müssen 0.08071025020177562 L Säure und 0.9192897497982244 L Wasser gemischt werden.
 Für 1 L einer Säure von 5 mol/L müssen 0.4035512510088781 L Säure und 0.5964487489911219 L Wasser gemischt werden.
 Für 1 L einer Säure von 10 mol/L müssen 0.8071025020177562 L Säure und 0.19289749798224376 L Wasser gemischt werden.
 Eine Säure mit 15 mol/L kann nicht aus einer Lösung mit 12.39 erstellt werden.

1.2 Aufgabe 2: Spektrum analysieren

Für die Übung ist ein IR-Spektrum von Dimethylacetamid (DMAc) gegeben.



###

Daten Laden Für das Laden der Daten ist eine Funktion vorgegeben, die die Daten des Spektrums als Listen mit den x- und y-Werten lädt. Nutze die Funktion, um die x- und y- Werte des Spektrums aus der Datei `dmac.csv` zu laden.

```

[2]: def load_spectrum(filename, separator=","):
      """
      Lädt ein .csv Spektrum als Liste
      filename: String mit Dateiname
      separator: Trenner der x,y Daten in der Datei
      Output:

```

```

x: Liste der Wellenzahlen
y: Liste der Intensitäten
"""

x_list = []
y_list = []
with open(filename) as file:
    for line in file:
        x, y = line.split(separator)
        x_list.append(float(x))
        y_list.append(float(y))

    return x_list, y_list

filename = "dmac.csv"
x, y = load_spectrum(filename)

```

1.2.1 Finde die Position des größten Peaks

Schreibe eine Funktion die das Absorptionsmaximum und die Wellenzahl des Absorptionsmaximum findet.

1.2.2 Hinweise

- Es wird eine **if/else** Bedingung benötigt
- Es kann gleichzeitig über den Index einer Liste und die Elemente geloopt werden:

```
for idx, element in enumerate(some_list):
```

- der Index `max_idx` kann am Ende genutzt werden um die entsprechenden Elemente der Listen aufzurufen

```

[3]: def peak_maximum(x, y):
    """
    Findet den x- und y- Wert für das Maximum der y-Werte
    x: Liste der Wellenzahlen
    y: Liste der Intensitäten
    Output:
    x_max: Wellenzahl des Absorptionsmaximum
    y_max: Absorptionsmaximum
    """

    max_idx = 0 # Index des größten Elements
    max_y = 0 # Startwert, das Maximum ist größer als 0
    for idx, element in enumerate(y):
        if element > max_y:
            max_y = element
            max_idx = idx
    x_max = x[max_idx]
    y_max = y[max_idx]

```

```

    return x_max, y_max

x_max, y_max = peak_maximum(x, y)

print(f"Das Maximum des größten Peaks liegt bei {x_max:.3f} cm-1")

```

Das Maximum des größten Peaks liegt bei 1642.000 cm⁻¹

1.2.3 Finde die Positionen der anderen markierten Peaks in der Abbildung

Auch wenn das Spektrum mehrere Peakmaxima aufweist, kann die Funktion `peak_maximum` nur das größte im Spektrum finden. Da wir aber wissen **wo** die verschiedenen Maxima auftauchen, können wir den Suchbereich eingrenzen um die gewünschten Peaks zu erhalten. Wir möchten die exakten Positionen von 4 Peaks finden: - Zwischen 900 cm⁻¹ und 1100 cm⁻¹ - Zwischen 1100 cm⁻¹ und 1300 cm⁻¹ - Zwischen 1300 cm⁻¹ und 1500 cm⁻¹ - Zwischen 1600 cm⁻¹ und 1800 cm⁻¹

Die Positionen der Peaks kann bestimmt werden, indem zunächst das Spektrum mit der Funktion `mask_spectrum` auf einen Teilbereich eingegrenzt wird, um danach innerhalb dieses Teilbereichs mit `peak_maximum` das Maximum zu finden.

1.2.4 Hinweise:

- List comprehensions können nützlich und effizient sein:

```
some_list = [element for element in Liste if Bedingung]
```

- Über mehrere Listen kann mit der `zip` Funktion geloopt werden:

```
for el1, el2 in zip(list1, list2):
```

```

[15]: def mask_spectrum(x, y, x_min, x_max):
    """
    Entfernt den Teil des Spektrums der nicht zwischen x_min und x_max liegt
    x: Liste der Wellenzahlen
    y: Liste der Intensitäten
    x_min: Kleinster X-Wert im Ausschnitt des Spektrums
    x_max: Größter x-Wert im Ausschnitt des Spektrums
    Output:
    x_masked: Wellenzahlen im Bereich x_min bis x_max
    y_masked: Absorption im Bereich x_min bis x_max
    """

    x_masked = [el for el in x if el > x_min and el < x_max]
    y_masked = [el2 for el1, el2 in zip(x, y) if el1 > x_min and el1 < x_max]

    return x_masked, y_masked

peak_x_min = [900, 1100, 1300, 1600]

```

```
peak_x_max = [1100, 1300, 1500, 1800]
n_peaks = len(peak_x_min)

for i, x_min, x_max in zip(range(n_peaks), peak_x_min, peak_x_max):
    x_masked, y_masked = mask_spectrum(x, y, x_min, x_max)
    x_peak, y_peak = peak_maximum(x_masked, y_masked)
    print(f"Peak {i} has its maximum at {x_peak} cm-1")
```

Peak 0 has its maximum at 1010.0 cm⁻¹

Peak 1 has its maximum at 1186.0 cm⁻¹

Peak 2 has its maximum at 1392.0 cm⁻¹

Peak 3 has its maximum at 1642.0 cm⁻¹